SupportDesk AI — Chat And Auto-Triage Any Customer Complaint

> **Short description:** A 2-agent terminal support desk in Python: a router sends each message to a chat Concierge or a Triage agent that turns complaints into validated support tickets. Raw OpenAI SDK, no framework.

Tech: Python · OpenAI API · Pydantic · python-dotenv · **Type:** Multi-agent (2 agents + router) · **Level:** Beginner

Overview

A 2-agent support desk that runs in your terminal. A **Concierge** agent chats naturally and remembers the conversation. A **Triage** agent turns a messy, free-text complaint into a clean, validated support ticket. A **router** decides, for every message, which agent should handle it — so the user never has to pick a mode.

Built with Python and the raw OpenAI SDK — no agent framework, on purpose.

Why

A real support inbox mixes casual questions with genuine complaints. Sorting every message by hand is slow and doesn't scale; making users pick a category ("is this a complaint?") pushes the work onto them. SupportDesk AI sorts each message automatically, chats when that's what's needed, and files a structured ticket when something is actually wrong.

Features

- **Streaming chat with memory** — the Concierge replies word by word and remembers earlier turns.
- **Structured ticket extraction** — the Triage agent returns a validated `SupportTicket`: customer name, issue category, urgency, one-line summary.
- **No invented data** — if the customer never gives a name, `customer_name` stays empty instead of being guessed.
- **Automatic routing** — a small model call picks the agent for each message and gives a reason.
- **Every routing decision is logged**, with its reason, so a misrouted message can be explained.

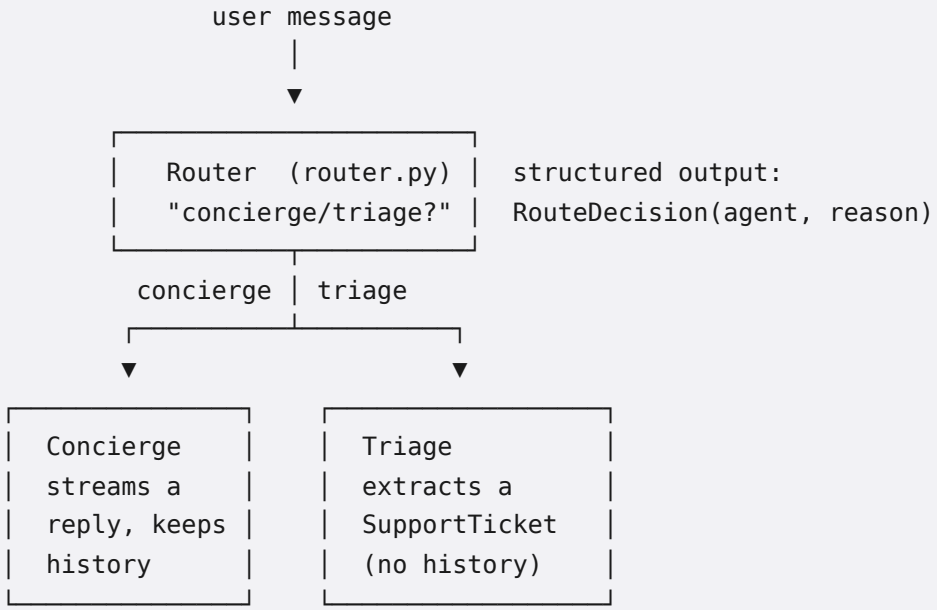
- **Long conversations are trimmed** before they reach the model's limit — the oldest turns go first, the system prompt always stays.
- **Fails cleanly** on a bad API key, rate limits, a too-long message, and invalid structured output.
- **Prompts live in plain text files**, so agent behavior can be changed without touching code.

Demo

```
$ python main.py
SupportDesk AI — type a message, or /quit to exit.
You: hi! what are your support hours?
Concierge: Hi there! Our support team is available Monday to Friday,
9am to 6pm. Anything I can help you with?
You: Hi, I'm Sarah Khan. I was charged twice this month, fix it today.
Ticket created:
  customer: Sarah Khan
  category: billing
  urgency:  high
  summary:  Customer was charged twice this month and wants it fixed.
You: /quit
```

(Log lines are printed too; they are left out here to keep the demo short. Model replies vary from run to run.)

Architecture



- The two agents never talk to each other; the router is the only coordination point.
- Concierge is **stateful** (it keeps the conversation). Triage is **stateless** (each complaint is judged on its own).
- Both the router and Triage use OpenAI structured outputs with Pydantic schemas, so their answers are checked, not parsed by hand.
- `services/desk_service.py` holds the router → agent flow, separate from terminal input/output, so the same flow could sit behind a web API.

Project Structure

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── data/
│   └── sample/
│       └── messages.json       test complaints + routing cases
├── scripts/
│   └── triage_samples.py       runs Triage on every sample message
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── concierge.py    concierge_agent() -> str
│   │   ├── router.py       decide() -> RouteDecision
│   │   └── triage.py       triage_agent() -> SupportTicket | None
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py          create_client(), send/stream_message()
│   ├── prompts/
│   │   ├── concierge.txt   the Concierge's system prompt
│   │   ├── router.txt      the router's system prompt
│   │   └── triage.txt      the Triage agent's system prompt
│   ├── services/
│   │   ├── __init__.py
│   │   └── desk_service.py handle_message(): router -> agent
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── history.py      estimate_tokens(), trim_history()
│   │   ├── logger.py       get_logger()
│   │   └── prompts.py      load_prompt(name)
│   ├── __init__.py
│   ├── config.py           Config, load_config()
│   ├── exceptions.py       MissingConfigError
│   └── schemas.py          SupportTicket, RouteDecision
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py real API calls
```

```
│   └── unit/
│       ├── test_config.py      no API key needed
│       ├── test_history.py     no API key needed
│       └── test_schemas.py     no API key needed
├── .env                        your real key — never committed
├── .env.example                the setting names, no real values
├── .gitignore                  keeps .env, .venv/, __pycache__/ out of git
├── LICENSE                     lets others reuse your code (MIT)
├── main.py                     the terminal loop — the file you run
├── README.md                   the repo's front page
└── requirements.txt            the 3 libraries, for pip install -r
```

Getting Started

Requirements: Python 3.10+ and an OpenAI API key.

```
git clone <this-repo-url>
cd SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # then put your key in .env
```

Configuration

Settings are read from `.env` (never committed):

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

The model (`gpt-4o-mini`) is set in one place: `MODEL` in `src/models/llm.py`. Agent prompts are in `src/prompts/`.

Usage

```
python main.py
```

Type any message. Small talk and questions go to the Concierge; problem reports become tickets. Type `/quit` to exit.

To see the Triage agent's ticket for every sample complaint:

```
python -m scripts.triage_samples
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key, no network, under a second
python -m tests.unit.test_config
python -m tests.unit.test_history
python -m tests.unit.test_schemas

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_agents
```

Every test case prints `OK` or `WRONG`. The unit tests check the config, history trimming and the schemas. The integration tests check a first reply, the startup key check, conversation memory, ticket extraction (including that no customer name is invented), and routing on clear messages.

Error Handling

Situation	What happens
`OPENAI_API_KEY` missing	Exits at startup with one clear line
Invalid API key	Found at startup with a 1-token call; exits cleanly
Rate limit	The OpenAI client retries twice with backoff; then the user is asked to wait
Conversation too long	Trimmed before sending; a single too-long message is rejected and removed
Invalid structured output	Caught; the user is asked to rephrase — no bad ticket is created
Router can't decide	Falls back to the Concierge, so no wrong ticket is filed

Design Decisions

- **Raw SDK, no framework** — every call is visible; nothing happens that the code doesn't show.
- **Two small agents, not one big prompt** — one prompt doing open chat *and* strict extraction tends to do both worse.
- **Model-based router with a reason** — handles wording a keyword list would miss, and the reason makes every decision explainable in the logs.
- **Concierge as the safe default** — a routing failure leads to a chat reply, never a false ticket.
- **Token estimate instead of a tokenizer** — about 4 characters per token, with a budget far below the real limit, so no extra dependency is needed.
- **`main.py` at the root, the code in `src/` beside it** — everything runs from the repo root: `python main.py` for the app, `python -m` for the tests and the script.

Limitations and Roadmap

- Memory lasts one session; nothing is saved to disk.
- Tickets are printed, not stored or sent anywhere.
- The router has two outcomes; an "ask a clarifying question" path would help on unclear messages.
- Possible next steps: save tickets to a database, add a web API, add CI to run the unit tests on every push, add tools for account lookups.

License

MIT — see `LICENSE`.